Listing 5.2 modifies the server code to only delete posts if the username is `owner`. First you'll add the cookie-parser middleware to the server. When the server receives a delete request, the username is pulled from the cookie and compared against the string `owner`. If the values match, the post is deleted. Otherwise, the server returns a 403 error (indicating that a user doesn't have permission to perform an action).

**Listing 5.2 Modifying the server code to read the cookie**

```
var express = require('express');
var cookieParser = require('cookie-parser');

var SERVER_PORT = 9999;
var serverapp = express();
serverapp.use(cookieParser);
serverapp.use(express.static(__dirname));
serverapp.use(handleCors);
serverapp.get('/api/posts', function(req, res) {
  res.json(POSTS);
});
serverapp.delete('/api/posts/:id', function(req, res) {
  if (req.cookies['username'] === 'owner') {
    delete POSTS[req.params.id];
    res.status(204).end();
  } else {
    res.status(403).end();
  }
});
serverapp.listen(SERVER_PORT);
```

**Adds cookie-parser middleware to server**

**Checks username to see if it's the owner**

**If user is valid, deletes post**

**...otherwise, returns a 403 forbidden error**

**NOTE** You may also have to install the cookie-parser middleware for this sample to work. You can do so by running `npm install cookie-parser`.

With this code in place, restart the web server and visit http://127.0.0.1:9999/login.html. Enter the username `owner` and any password (remember you aren't validating passwords because this is a sample), and click the Login button. Once you're redirected to the blog, click a Delete link.

**NOTE** The login page resides at 127.0.0.1:9999 and not localhost:1111. The login page from localhost:1111 will not set the correct cookie.

After clicking the Delete link, you'd expect the request to succeed. The previous section verified that cookies are included on API requests, but the sample will once again fail with the error shown in figure 5.3. Switching over to the Network tab as shown in figure 5.4 reveals that the cookie isn't included in the request.

Where is the cookie you just set? This is once again a distinction between same-origin and cross-origin requests. Cookies are always included with same-origin requests, but they aren't included on cross-origin requests by default. Because clicking the Delete link triggers a cross-origin request, the request goes through but the cookies don't. Luckily CORS offers a solution for including cookies with requests.